

Politechnika Warszawska

Wydział Matematyki
i Nauk Informacyjnych

Ensemble Learning – Research Project

Project 1: the report in plain words

Non-linear and Interaction Features for Deep Forest

Authors:

Wojciech Jasiński

Sandra Adamiec

Warsaw, 12 November 2026

What this is. A companion to the short report. It says the same things in everyday words, for a reader who got stuck somewhere. Section numbers and paragraph titles are the same as in the short report, so you can look up the place where you got stuck. Figures and tables are not repeated: "Figure 2" means Figure 2 of the short report. There is nothing new here, and where the two texts seem to disagree, the short report is right. A list of the words we use is at the end.

Abstract

A Deep Forest is a stack of random forests. The first layer looks at the data and makes a guess. Every later layer looks at the data again, together with the guess of the layer before, and tries to do better. We asked one question: does a layer do better if we hand it a few extra, ready-made columns? For example two columns multiplied together, or a column that says how unsure the previous layer was. Before running anything we wrote down five guesses about what would help (the hypotheses) and the rules for judging them. Then we tried 11 methods on 32 datasets.

Main findings

- Nothing we added made the model better. If some gain exists, it is tiny: a model that is wrong 10 times in 100 would, at best, be wrong about 9.93 times.
- The idea is not nonsense. On made-up data where the answer really is two columns multiplied, the method finds those two columns and removes about 40% of the errors.
- Why it fails on real data, as far as we can tell: every extra column does a little harm, most of all one filled with junk, and on real data a multiplied column does not help enough to make up for it. This comes from extra experiments we thought of afterwards, so it is a hint, not proof.
- Of all 11 methods, a tuned LightGBM (a different, popular tree model) did best. The plain Deep Forest did beat a single random forest as big as one of its layers.

1 Introduction

A decision tree answers by asking a chain of yes/no questions, each about one column: "is the age above 40?", then "is the income below 3,000?", and so on. A random forest is a few hundred such trees, each built a little differently, that vote. A Deep Forest puts forests in layers. Layer 1 sees the data. Layer 2 sees the data and layer 1's guess, so it can correct it, and so on. The final answer comes from the layer that scored best during training.

This has two weak spots. First, the only new thing a layer learns from the one before is its guess. Second, a tree asks about one column at a time, so it struggles when two columns matter only together. Say the answer is "yes" whenever two numbers have the same sign. Knowing only the first number tells you nothing, and neither does the second. But multiply them, and one question is enough: is the product above zero? That multiplied column is what we call a *product*. The course topic asked for exactly this sort of thing: new columns made from one column in a curved way ("non-linear") or from two columns together ("interaction").

> **How to read Figure 1.** Left to right: the data goes into the first layer of forests (the small trees), and what comes out goes into the second. Grey squares are the original columns, blue ones are a layer's guesses, and orange ones are the columns we add.

Related work. Other people have changed what one layer passes to the next. The three papers we cite either replace the guesses with something else, build new columns with a heavy search method, or change several things at once. Our way of picking products is borrowed from an older idea: first see what a simple model gets wrong, then add only the products that line up

with those mistakes. What is new in our work is testing such columns on their own, with fair comparisons next to them.

The short report is itself a summary. The plan and the long results report have the details.

2 What we tested

Every variant is the same plain Deep Forest (the *base cascade*) plus one kind of extra column. Nothing else changes, so any difference can only come from that column. The five guesses:

H1 Feature products help. Multiply two columns. Keep the pairs whose product lines up best with what the previous layer got wrong. We expected fewer errors.

H2 The choice matters. Picking pairs this clever way should beat (a) picking pairs blindly and (b) picking pairs whose product lines up with the answer itself.

H3 Feature–prediction products help. Multiply a column by the previous layer's guess.

H4 Uncertainty helps. Tell the next layer how unsure the previous one was.

H5 Monotone activations help. Pass the guesses through a squashing function before handing them on.

The added columns. Before multiplying, every column is put on the same scale (*standardized*), so a column measured in thousands does not drown one measured in fractions. Products of two columns (XX) can be made from the start, so they go into every layer. Everything that needs a guess (XP products, uncertainty, the squashing function) starts at layer 2, because layer 1 has no earlier guess to use. K is how many products we add: about as many as there are original columns, fewer when there are only a few. The names:

- *Residual-XX*, our main idea. The *residual* is the true answer minus the guess, so simply the mistake. This variant picks the products that follow the mistakes most closely, and picks again at every layer.
- *XP-MI* picks by *mutual information*: a score for how much knowing a column narrows down the answer.
- *Non-linear* adds three columns: how unsure each of the two kinds of forest is, and how much the two kinds disagree. Think of how split a vote was.
- *Monotone* adds a *sigmoid* of each guess. A sigmoid is an S-shaped function that squeezes numbers into the range 0 to 1 and never changes their order.

How to read Table 1. The top eight rows are versions of our Deep Forest. The bottom three are outside yardsticks: one big random forest, the same forest with products, and LightGBM. A "control" is a comparison that is there to rule out a boring explanation. If randomly picked products did as well as cleverly picked ones, the clever picking would not be what matters.

3 How we tested

Cascade. "Cascade" is the stack of layers. We copied the settings of the official Deep Forest package (DF21): four forests per layer, 100 trees each, and layers are added until two in a row stop helping. We changed two things on purpose. First, when a layer makes guesses to pass on, a forest must not guess rows it was trained on, because it would look better than it is, like marking your own exam. So the training data is cut into three parts and each part is guessed by forests that never saw it. These parts are the *inner folds*. Second, the best layer is picked by a score that treats all classes equally. The package cannot take extra columns, so we rebuilt the cascade ourselves. Our copy is close to the package but not identical (the same on two test datasets, a little worse on two others), so strictly speaking our results are about our copy.

Data. 32 public tables. In half of them the task is to pick a class (classification), in the other half to predict a number (regression). We threw out columns that would let a model cheat, such as row numbers, dates and anything that gives the answer away. Empty cells were filled in, categories were turned into 0/1 columns, and numbers were put on the same scale. "Fitted on the training rows only" means the test rows had no say in any of this, so there was no peeking.

Runs. To test a method fairly, cut the data into five parts (*folds*), train on four, test on the fifth, and take each part as the test part once. We did this three times with three different cuts, which gives 15 scores per method on each dataset. Rows of the same patient always go together: otherwise the model could recognise the person, not learn the pattern. All methods get the same cuts. The *error* is what we measure: in classification, the share of wrong answers, counted per class so that a rare class matters as much as a common one; in regression, the typical size of a miss (RMSE). We did not hunt for good settings for any variant. Only LightGBM was tuned, so that the outside yardstick is a strong one. The main experiment is 32 datasets $\times$ 11 methods $\times$ 15 = 5,280 runs. The *sensitivity checks* change one setting at a time to see whether the story changes.

Deciding a hypothesis. For each dataset we ask by how much the variant changed the error, in relative terms. If the error drops from 10% to 9.5%, that is a reduction of 5%. A minus sign means the variant was worse. This gives 32 numbers per comparison, one per dataset. If an extra column did nothing, the pluses and minuses would look like coin flips. The *Wilcoxon test* asks whether there are too many pluses, and too big, for coin flips. Its answer is the *p-value*: how often pure luck would give a result at least this lopsided. Below 0.05 we count it as real. Since we run six tests, we have six chances to get lucky, so *Holm's correction* raises the bar for each. A hypothesis is supported only if the test passes and the middle dataset (the *median*) actually improved.

4 Results

4.1 At a glance

None of the five guesses came true. In every comparison, some datasets got better, some got worse, and the typical change is under 1%. The measurements are precise enough to say more than "we saw nothing": a typical gain above about 0.7% would be surprising given what we measured.

How to read Table 2. Take the first row. "14/18/0" means residual-XX was better than the base cascade on 14 datasets and worse on 18. The median of -0.66 means the middle dataset got 0.66% worse. The last column is the p-value, which would have to be below 0.05. The closest any row comes is 0.12.

How to read Figure 2. Each dot is one dataset. Right of zero, the first method won; left of zero, it lost. In every row the dots sit on both sides, bunched around zero. The black mark is the typical change, and the bar through it shows how sure we are of it. One dot is far off to the left (Tic-Tac-Toe). There both errors are tiny, about 1.4% and 2.3%, so a small difference looks huge in relative terms.

4.2 H1: Do feature products help?

No. With the products the model was a little worse on 18 of the 32 datasets. Looking at each dataset on its own, it was clearly better on one and clearly worse on ten; on the rest we cannot tell. The products are not worthless, though. Section 6 explains why they still lose.

4.3 H2: Does the way of choosing products matter?

Hardly. Clever picking did no better than blind picking. Against picking by the answer itself it won more often than it lost, but not convincingly: the result fails the stricter bar and shows clearly in only one of our three repeats. Its biggest win, on the Bike Sharing dataset, is really the other method failing. That method keeps the same products in every layer, and there it did 15% worse than adding nothing. And adding nothing was better than any way of picking.

4.4 H3: Do feature–prediction products help?

No. The typical change is about zero. On six datasets literally nothing changed, mostly because the best layer there was always layer 1, and these columns only start at layer 2, so they were never used. In regression the variant was more often worse than better (a hint only; this was not one of the planned tests). It is also slow: a typical run takes 46% longer.

4.5 H4: Do uncertainty columns help?

They do nothing we can see. About as many datasets got better as got worse, and the middle one did not move. The columns cost almost no time, and adding them together with the products did not clearly help either.

4.6 H5: Do monotone activations help?

No, and there is a side effect. Our main score only checks whether the right class was picked. Two other scores, ROC AUC and log-loss, check how good the stated probabilities are, for example whether "70% sure" is right about 70% of the time. These got worse on 14 and 15 of the 16 classification datasets.

Our guess at the reason: before building trees, the forests sort every column into bins by rank. A sigmoid never changes the order of values, so after sorting into bins the new column is the same as the old one: a photocopy. It is like recording temperature in Celsius and again in Fahrenheit. "Warmer than 20 °C?" and "warmer than 68 °F?" split the days in exactly the same way. With copies of the guesses in the data, the forests probably lean on the guesses too much and become too sure of themselves. This was not a planned test, so treat it as a hint. With the binning switched off, none of the three functions we tried (sigmoid, ReLU, GELU) helped either.

4.7 Are the results robust?

"Robust" means the conclusion survives when we look at the data in another way. As far as we checked, it does.

- *By task:* looking at classification alone or regression alone changes nothing, and neither does using other scores (Table B1).
- *At a fixed depth:* we forced every model to stop at layer 1, 2 or 3, so that the choice of depth cannot hide anything. Still no variant beats the plain model. Picking products by the mistakes still beats picking by the answer.
- *Depth rule:* in 43% of runs the model answers from layer 1, before three of the four kinds of column have even been added. The fixed-depth check shows that no gain is hiding behind this. It did show that our stopping rule is too cautious: always using two layers would have been better on 19 of 32 datasets.
- *Other settings:* half or double the number of products, more forests, more trees, no binning, five inner folds. The story stays the same.

4.8 How do the other models compare?

On each dataset we order the 11 methods from best (rank 1) to worst (rank 11) and average the ranks over the datasets. LightGBM comes out best. It beats the plain Deep Forest on 21 of 32 datasets, although by our test this could still be luck. The plain Deep Forest beats one big random forest with as many trees as one layer has, so stacking layers does pay off. Products do not help a single forest either: the forest with products is worse on 24 of 32 datasets.

How to read Figure 3. Further left is better. Rank differences smaller than the shaded band could be luck, so any method inside the band cannot be told apart from the plain Deep Forest. All our variants are inside it.

5 Does the idea work at all?

Yes, on data made for it. Before you declare a field free of metal, bury a coin and check that your detector beeps. This is called a *positive control*. We made two small artificial datasets (300 rows, six random columns) in which the answer really is a product. In the first, the class is "do x_1 and x_2 have the same sign?", with one answer in ten flipped on purpose, so nobody can score above 90%. In the second, the target is $x_1 + 3x_2x_3$ plus some noise. Here the products removed about 40% of the errors, and the right pair of columns was found at the first layer every time. We then made it harder: 1,000 rows and 30 columns, nearly all of them junk. The right pair was still found every time, the error more than halved, and the method did better than LightGBM. So the detector beeps.

How to read Figure 4. Left panel: share of right answers, longer is better. Right panel: size of the miss, shorter is better. The blue methods are the ones that add products. On this data they are clearly ahead.

6 Why the products do not help on real data

We ran these experiments after seeing the results, to look for a reason. They are detective work after the fact, mostly repeated only once, so they give a plausible story and not proof. To stay honest, we wrote down before each run what result would count as a "yes".

Extra columns cost more than the products bring. We took the chosen product columns and dealt their values out to the wrong rows. They hold the same numbers but now mean nothing: junk that takes up as much room. With junk added, the model got worse on 26 of 32 datasets, typically by 3%. That is the price of extra columns that carry nothing. Real products beat the junk on 25 datasets, so they are better than junk. But products picked blindly beat the junk just as often (26 datasets). So this does not show that our chosen products add something the model lacks, only that a real column hurts less than a junk one. What remains is a small loss. We do not know exactly why extra columns hurt. We checked two easy explanations and both failed: it is not that the new columns push the original ones out when a tree decides which columns to try, and it is not the way we centre columns before multiplying.

How to read Figure 5. Top bar: junk columns against the plain model. Middle bar: real products against junk. Bottom bar: real products against the plain model. The three numbers do not add up exactly, because each is the middle value of a different list.

A weak interaction is enough. Maybe the method only notices very strong products? We took 13 real datasets, added two random columns, and mixed their product into the target at a strength we chose: none, weak (6% of the target's ups and downs), medium (20%) or strong (50%). This is the easiest possible case for the method. Before the run we said that "products help" means better on at least 9 of the 13 datasets and better in the middle one. The products

helped already at the weak setting (11 of 13), and more as the strength grew. We had expected them to need the medium setting. So the method is not blunt: a clean product gets found even when it is weak.

How to read Figure 6. Left to right, the planted product gets stronger; higher means the products help more. The two lines lie almost on top of each other: one big forest gains as much from products as the Deep Forest does.

A measure of interaction does not predict where products help. Can we tell in advance which datasets have columns that work together? A rough way: compare a model built from one-question trees, which cannot combine columns, with one built from deeper trees, which can. The more the deeper trees gain, the more the columns seem to interact. We hoped products would help most where this score is high. They did not; there is no link. One possible reading: real data does have columns that work together, but not as neatly as "one times the other".

Depth already does part of the job. On 7 datasets the products clearly improved the first layer. On 6 of those, the plain model's later layers improved it by even more, with no products at all. So the layers already repair much of what the products would repair.

7 Conclusions

Extra columns between the layers did not make Deep Forest better on our 32 datasets, and if a gain exists it is well under 1%. The idea itself works: where the answer truly contains a product of two columns, the method finds it. On real data the extra columns cost a bit more than they bring, and the layers already do part of the same job. Picking products afresh at every layer lost less than keeping the same ones, but was no better than picking blindly, and adding none was best.

Limitations.

- Everything in Section 6 was thought of after the results and mostly run once. If you look in enough places you will find something by luck, and most results outside the six planned tests were not given a stricter bar for that.
- When both errors are near zero, a relative change looks dramatic (Tic-Tac-Toe). Our test works on the order of the changes, not their size, so one such dataset cannot dominate it.
- On three datasets we treated number-coded categories as plain numbers, which the plan said not to do. We redid those three properly and no conclusion changed (one repeat).
- We only add columns between layers. The instructor suggested interactions inside a layer. We have not tried that.

Further work.

- Interactions inside a layer, once we agree what exactly that means.
- Adding products only where they beat junk columns on data kept aside, so the cost is paid only when there is something to gain.
- A better rule for when to stop adding layers. The current one stops too early, and five inner folds instead of three made every cascade we tried more accurate.

8 Reproduction

The code, all settings and the summaries are public at github.com/wojtke/deep-forests. Three commands download the data, run an experiment and produce its tables and figures. A *seed* is the number that fixes every random choice, so anyone who reruns the code gets the same cuts

of the data. The planned results were made with the first version of the code in the repository; the code for the extra experiments was added later. One thing is not published yet: the large files with each layer's guesses for every run.

9 AI assistance

An AI coding assistant (Claude Code) wrote the experiment code following the plan, ran the experiments, suggested the extra experiments and drafted the reports, this text included. Other AI reviewers checked the code, the results and the text. The authors decided what to run and checked the results and the text.

Appendix tables

Table A1 lists every setting and where it comes from: the official package (DF21), the plan, or our own choice. **Table A2** lists the 32 datasets with their sizes. **Table B1** repeats the six comparisons for classification and regression separately and with other scores. **Table B2** gives each method's average error on each dataset. The number after ± (the standard deviation) says how much the 15 runs differ from one another.

Words used

Balanced accuracy	Share of right answers, worked out for each class and then averaged, so a rare class counts as much as a common one.
Base cascade	Our plain Deep Forest, with no extra columns. Everything is compared with it.
Binning	Sorting a column's values into groups by rank before the trees are built. It makes forests faster.
Cascade	The stack of layers.
Control	A comparison that is there to rule out a boring explanation.
Feature	A column of the data. A prediction is a layer's guess.
Fold	One of the parts the data is cut into for testing. Inner folds are the same trick inside the training data.
Holm's correction	A stricter bar for each test, needed because several tests give several chances to get lucky.
Median	The middle value when the datasets are put in order.
Mutual information	A score for how much knowing a column narrows down the answer.
p-value	How often pure luck would give a result at least this lopsided. Below 0.05 we count the result as real.
Positive control	A test on data where we know the method should work.
Product (XX, XP)	Two columns multiplied. XX: two features. XP: a feature and a prediction.
Rank	A method's place on one dataset, 1 for the lowest error.
Relative error reduction	By what share the error went down. From 10% to 9.5% is a reduction of 5%.
Residual	The true answer minus the guess: the mistake still left.
RMSE	The typical size of a miss in regression. Big misses count extra.
Seed	The number that fixes the random choices. Another seed is another repeat of the same experiment.
Shuffled products	Product columns with their values dealt to the wrong rows: the same numbers, no meaning.
Wilcoxon test	Asks whether one method wins more often, and by more, than luck would explain.